Radar Telemetry Over LoRa

1. Project Charter

This project came from work, where the radar team has limited access to the ground control station during integration events and flight testing. Because of that problem, I will build a simple LoRa telemetry system that gives another way to gather radar status data during testing. I will use one Heltec board as the air Heltec and a second Heltec board as the ground Heltec. I will use one laptop on the air side and one laptop on the ground side so the setup better matches the way the system would be used during experiments. The ground side will send commands, and the air side will return updated radar status over the LoRa link.

I will simulate radar behavior instead of using actual radar hardware because the hardware is not available for this project. The simulated system will support radar modes and functions that are closer to the real system, including MTI, SARVideo, Maritime Large Vessels, Maritime Small Vessels, and transmission enable or disable. The operator will be able to change those modes from the ground side while the telemetry system reports the commanded and received radar status. This keeps the technical work focused on communication, monitoring, and troubleshooting instead of difficult hardware access problems.

I will build the project in stages so each subsystem can be tested before full integration begins. First, I will confirm serial communication between the laptops and the Heltec boards under controlled bench conditions. Next, I will confirm packet creation and LoRa transmission between the two Heltec boards. After that, I will confirm full two-way communication, where the ground side sends a radar command and the air side returns updated radar status. Last, I will confirm live

display and logging on the ground laptop during one end-to-end test. This staged approach lowers risk because I can find problems early and fix them before combining the full system.

My minimum viable product will be a stable telemetry demonstration that includes bench testing and motion testing during a slow car drive while packets are being transmitted. I will consider the minimum viable product complete when one continuous test runs without resets and shows matching commanded and reported data on the ground laptop. At startup, the system will send one full status packet so the ground side receives the current radar condition. After that, the system will send updated telemetry when the radar mode or function changes and when the ground side sends a new command. The ground side must display matching information clearly enough for an observer to understand the current system condition. I will use a simple Python display first because it is faster to build and easier to debug than a more advanced dashboard. This choice keeps the project realistic and gives me a better chance of finishing on time.

Bench testing will be the starting point, but I will place more emphasis on testing and documenting how the system behaves under different conditions. I will document how the system performs during bench testing, during motion, and at different distances when possible. I will also document whether the link stays stable while the system is moving. This added testing will help show not only that the system works, but also how well it works. The main promise for this quarter is still a simple and achievable end-to-end telemetry system, but stronger testing and documentation will improve the final quality of the project.

After the minimum viable product is complete, I will explore several routes that can improve the final quality of the project. These routes may include packet counters, packet loss tracking, saved CSV logs, improved HMI displays, additional distance testing, and more detailed motion testing. I do not need to complete all of these routes for the project to succeed. The minimum viable

product is the required baseline, while the added testing and feature development will improve the final result. This keeps the project well scoped while still leaving room for a stronger final outcome.

The biggest project risks are time limits, LoRa setup problems, serial communication errors, and delays from building a display that is too complex. I will reduce those risks by keeping the packets short, using a simple Python display, and removing lower-priority features if milestones slip. Another risk is that the air and ground systems may not match during early testing because of timing or format errors. I will reduce that risk by testing each step separately before the full radio link is used. I already have the main hardware and software tools, which lowers cost and lowers setup risk. This combination gives the project a strong chance of finishing on time.

2. Group Management

I will be the only person working on this project, so I will handle the air Heltec, ground Heltec, integration, testing, documentation, and final presentation preparation. Because this is a one-person project, I will make the main technical and schedule decisions myself. If I need feedback, I will ask my instructor or classmates for suggestions, but I will remain responsible for the final direction of the project.

My wife and son will assist only with the data collection process during motion testing. Their role will be limited to driving slowly in a parking lot to simulate taxiing down a runway while I observe and collect telemetry results. They will not be involved in design, coding, testing decisions, documentation, or project management. Their support is only for safe and simple movement during the motion test portion of the project.

I will communicate project progress through GitHub and biweekly status updates. GitHub will be used to track code changes, setup notes, screenshots, logs, milestone writeups, and testing evidence. The biweekly status updates will be used to report progress, identify risks, and confirm whether the project is still on schedule.

I will know the project is off schedule when a planned milestone cannot be demonstrated with code, logs, screenshots, or video evidence. If that happens, I will remove lower-priority work and return focus to the minimum viable product. This structure keeps the project realistic and prevents optional work from delaying the core system.

3. Project Development

The main development roles are air-side firmware, ground-side firmware, display software, testing, and project documentation. The project will use two Heltec WiFi LoRa boards, two laptops, USB cables, PlatformIO, VS Code, Python, GitHub, and shared documents. I already have the main hardware, so the required project can be completed without major new purchases. A battery pack may be useful later, but it is not required for the promised work this quarter.

The air Heltec and ground Heltec firmware will be developed in C++ using PlatformIO inside VS Code. The air laptop radar simulator and the ground laptop display will be developed in Python because that supports fast serial testing and easy display changes. The packet format is based on radar modes and functions that are meaningful for troubleshooting, including MTI, SARVideo, Maritime Large Vessels, Maritime Small Vessels, and transmission enable or disable. This will make the final display more useful than a generic state monitor because it will help compare the commanded radar mode to the reported radar mode.

Testing will happen in layers so each subsystem can be verified before the full system is integrated. The first tests will confirm USB serial input from the ground laptop to the ground Heltec and from the air Heltec to the air laptop under stable bench conditions. The next tests will confirm LoRa transmission between the two Heltec boards at short indoor range. The next tests will confirm full two-way communication, where the ground side sends a command and the air side returns the updated radar status. The final tests will confirm live display, saved logs, and stable operation during bench testing and during motion testing in a slow car. I will also document how the link performs during motion and whether packets remain stable while the system is moving.

I will place strong emphasis on documenting the GitHub repository because it is an important part of the class deliverables. The repository will include code commits, setup instructions, packet format notes, testing procedures, screenshots, logs, milestone writeups, and README updates. Each major milestone will be supported by both working code and written documentation in the repository. This documentation will make milestone reviews clearer and reduce confusion during final report preparation and presentation work.

Testing Plan

The testing plan will measure how well the telemetry system performs during two-way communication and radar mode changes. I will test the system at 20-foot distance increments and record the results at each location. At each distance, I will command several radar modes and TX states, including MTI, SARVideo, Maritime Large Vessels, Maritime Small Vessels, transmission enable, and transmission disable. I will record whether the command was sent, whether the correct status was returned, whether the two-way communication was successful, the time delay,

the number of repeats needed, RSSI, and packet loss. These results will be used to make graphs that show communication success, packet loss, latency, and repeat count as distance increases.

At the time of resubmission, the two-laptop setup is operating and the next step is repeated motion and distance testing to collect data for graphs and documentation.

4. Project Milestones and Schedule

The first deliverable will be a finished system design with packet format, telemetry fields, repository structure, and a realistic eight-week schedule. The second deliverable will be a working one-way telemetry link that proves the basic communication path works. The third deliverable will be a working two-way LoRa link that allows the ground side to send radar commands and receive updated radar status from the air side. The fourth deliverable will be a working ground display that shows commanded and reported radar state and saves basic logs. The fifth deliverable will be a complete minimum viable product demonstration that proves the full telemetry path works from command input to final display during bench testing and car motion testing.

Week one will finalize scope, confirm hardware access, and create the repository structure for firmware and display work. Week two will define telemetry fields, packet format, software interfaces, and the exact success criteria for the minimum viable product. Week three will complete one-way serial and LoRa communication tests. Week four will complete two-way

command and returned status communication between the two laptops through the Heltec boards.

The target week for MVP completion will be Week 5. During Week 5, I will complete the full end-to-end two-way telemetry path and confirm that the system works during both bench testing and slow car motion testing. Week 6 will be used for improving the ground laptop display and organizing saved logs. Week s7 will focus on characterization testing, GitHub documentation, and collecting screenshots, logs, and short videos. Week 8 will finish documentation, finalize the presentation, and perform the final demonstration.

Each deliverable will require visible proof such as a screenshot, short video, packet log, GitHub commit history, or documented repository update. Deliverable one will be shown through the finished design document and packet format description. Deliverable two will be shown through serial output proving that one-way communication works. Deliverable three will be shown through matching ground-side commands and air-side returned radar status from a live test. Deliverable four will be shown through a working display window and saved telemetry output files. Deliverable five will be shown through one continuous demonstration video, final system screenshots, and documented testing results.

The highest-priority milestones are serial command input, telemetry packet creation, LoRa packet transfer, receiver forwarding, ground display, and GitHub documentation. Useful but lower-priority work includes cleaner graphics, packet counters, connection status indicators, and CSV export for later Power BI use. Optional work includes additional range testing, more detailed packet characterization, and later visualization improvements. This schedule remains realistic because the core system stays simple while the added testing and documentation improve the final project quality.

5. Excel-Ready Schedule

Weekly Milestones Schedule

Week	Milestone	Owner	Proof of Completion
1	Finalize scope and repository setup	Manny	Repository and task list
2	Complete system design and packet format	Manny	Design document
3	Complete one-way serial and LoRa communication test	Manny	Serial output screenshot
4	Complete two-way command and returned status communication	Manny	Sent packet log
5	Complete MVP, including end-to-end bench test and slow car motion test	Manny	End-to-end test notes, motion test notes, and demo log
6	Complete ground laptop Python display and saved logs	Manny	Live display screenshot and saved log file
7	Complete characterization testing and GitHub documentation	Manny	Complete characterization testing and GitHub documentation
8	Complete final demo and report	Manny	Demo video and report

Gantt Chart Schedule

Task	W1	W2	W3	W4	W5	W6	W7	W8
Scope and setup	■							
Design and packet format		■						

Packet creation, startup status, and 10-second updates	■	■				
MVP completion: bench test and car motion test		■	■			
Ground display and saved logs			■			
Documentation		■	■	■	■	■
Final demo and report						■